

Laboratorium 4

Apache Spark - platforma typu open-source do przetwarzania dużych ilości danych (typu big data) zbudowana dla szybkości, łatwości użycia i zaawansowanej analityki. Wykonuje obliczenia w pamięci w celu analizy danych w czasie rzeczywistym.

ZADANIA

Wykorzystując spark napisz program, który implementuje reguły asocjacyjne za pomocą algorytmu A-priori. Program powinien - na podstawie ostatnio przeglądanych przez użytkowników przedmiotów - identyfikować te, które regularnie występowały wspólnie w ramach pojedynczej sesji.

Algorytm powinien znaleźć wszystkie pary (3 pkt) i trójki (2 pkt) przedmiotów, które wystąpiły wspólnie w ramach pojedynczej sesji co najmniej 100 razy.

Plik wejściowy 4.txt:

- każda linia w pliku reprezentuje pojedynczą sesję użytkownika z rozdzielonymi spacjami ośmioznakowymi identyfikatorami przeglądanych produktów.

Rozpocznij od załadowania potrzebnych paczek:

```
from pyspark import SparkContext
import sys
```

Uruchom środowisko i wczytaj dane:

```
sc = SparkContext()
file = sc.textFile('4.txt')
```

(Pamiętaj aby po wykonaniu zadań zakończyć/zatrzymać środowisko komendą sc.stop())

AKTYWNY MOŻE BYĆ TYLKO JEDEN OBIEKT SparkContext()

W Spark najpierw trzeba zastosować mapowanie danych:

```
lines = file.map(lambda line: split_line(line))
```

gdzie np.:

```
def split_line(line):
    items = line.split(' ')
    while " " in items:
        items.remove("")
    return items
```

Wypisanie zawartości RDD:

```
from __future__ import print_function
lines.foreach(print)
```

Lub wypisanie pierwszych np. 10 wierszy:

```
for i in lines.take(10):
    print(i)
```

Częstotliwość przeglądania pojedynczych przedmiotów (interesują nas tylko te wyszukiwane co najmniej 100 razy) można uzyskać np. w następujący sposób:

```
singles = lines.flatMap(lambda x: x).map(lambda x: ((x,), 1))
singles = singles.reduceByKey(lambda x, y: x + y)
singles = singles.filter(lambda x: x[1] >= 100).collect()
```

```
for i in singles:
    print(i)
```

Tworzenie par i trójek można uzyskać np. przez zastosowanie metody `permutations()` z pakietu `itertools`

Przykładowy wynik dla **par** (pierwsze 10 wierszy):

```
DAI93865 [FRO40251]
GRO85051 [FRO40251]
GRO38636 [FRO40251]
ELE12951 [FRO40251]
DAI88079 [FRO40251]
FRO92469 [FRO40251]
DAI43868 [SNA82528]
DAI23334 [DAI62779]
ELE92920 [DAI62779]
DAI53152 [FRO40251]
```

Przykładowy wynik dla **trójek** (pierwsze 15 wierszy):

```
DAI23334 ELE92920 [DAI62779]
DAI31081 GRO85051 [FRO40251]
DAI55911 GRO85051 [FRO40251]
DAI62779 DAI88079 [FRO40251]
DAI75645 GRO85051 [FRO40251]
ELE17451 GRO85051 [FRO40251]
ELE20847 FRO92469 [FRO40251]
ELE20847 GRO85051 [FRO40251]
ELE26917 GRO85051 [FRO40251]
FRO53271 GRO85051 [FRO40251]
GRO21487 GRO85051 [FRO40251]
GRO38814 GRO85051 [FRO40251]
GRO73461 GRO85051 [FRO40251]
GRO85051 SNA45677 [FRO40251]
GRO85051 SNA80324 [FRO40251]
```